13 CIRCUITE LOGICE COMBINAȚIONALE

13.1 Minimizarea funcțiilor logice

Circuitele alcătuite din porțile logice de bază, a căror operare poate fi descrisă cu ajutorul algebrei Booleene, se numesc circuite logice combinaționale, deoarece *în fiecare moment de timp starea logică a ieşirii depinde de modul în care se combină nivelurile logice ale intrărilor în acel moment de timp*. Ele nu au capacitatea de memorare a informației.

Problema esențială care trebuie rezolvată cu ajutorul circuitelor logice combinaționale este implementarea unor funcții logice cu ajutorul *unui număr minim de porți logice*. Pentru atingerea acestui scop, funcția logică trebuie adusă la o formă cât mai simplă care să conțină un număr minim de termeni. Acest proces se numeşte *minimizarea funcției logice*. Despre funcțiilor logice aduse la o formă minimizată se mai spune că sunt scrise sub *formă canonică*. Există două forme canonice utile în proiectarea circuitelor logice combinaționale, *suma de produse* sau *produsul de sume,* prima dintre ele fiind cea mai folosită.

Minimizarea funcțiilor logice până la una din formele canonice se poate face în două moduri:

- folosind teoremele algebrei Booleene
- folosind tehnica diagramelor

În cazul scrierii funcției sub formă de sumă de produse, ea este alcătuită din doi sau mai mulți termeni care includ funcția ŞI, după care aceştia sunt uniți între ei cu ajutorul funcției SAU. Termenii ŞI ai sumei trebuie să respecte următoarea regulă:

> *un termen ŞI poate conține una sau mai multe variabile Booleene, variabile care pot fi prezente o singură dată, în forma normală sau complementară.*

Această regulă ne precizează faptul că semnul de inversiune poate să apară numai deasupra variabilelor individuale. De aceea nu sunt admişi în expresia unei funcții logice termeni de forma $\overline{ABC}$ sau $\overline{ABC}$.

13.1.1 Minimizarea algebrică

Minimizarea algebrică se poate realiza utilizând teoremele algebrei Booleene dar, din păcate, nu ştim întotdeauna care teoremă trebuie aplicată într-o situație dată şi dacă expresia obținută este sub cea mai simplă formă

posibilă. De aceea, mai ales în cazul funcțiilor complicate, simplificarea algebrică poate deveni o sursă de erori şi necazuri.

În general, în procesul de simplificare algebrică a unei funcții logice se recomandă efectuarea a doi paşi:

- funcția se scrie sub formă de sumă de produse
- termenii sunt grupați după factorul comun (dacă există), care apoi se scoate în fața parantezei. Această operație poate conduce la eliminarea unuia sau mai multor termeni.

Să aplicăm aceste etape la implementarea funcției:

$$z = ABC + A\overline{B}\left(\overline{\overline{AC}}\right) \qquad (13.1)$$

Funcția z poate fi simplificată cu ajutorul teoremelor algebrei Booleene. Astfel, factorul $\overline{\overline{AC}}$ poate fi scris:

$$\overline{\overline{AC}} = \overline{\overline{A}} + \overline{\overline{C}} = A + C$$

astfel încât funcția z devine:

$$z = ABC + A\overline{B}(A + C) = ABC + A\overline{B}A + A\overline{B}C$$

Deoarece $AA = A$:

$$z = ABC + A\overline{B} + A\overline{B}C = AC(B + \overline{B}) + A\overline{B}$$

Dar $B + \overline{B} = 1$, astfel încât se obține forma minimizată sub formă de sumă de produse a funcției z:

$$z = A\overline{B} + AC \qquad (13.2)$$

Pe baza acestei expresii se poate proiecta circuitul cel mai simplu care să o realizeze, circuit prezentat în fig. 13.1.

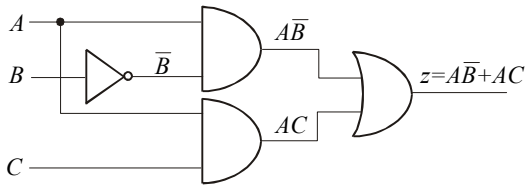


Fig.13.1

Pentru a vă convinge că munca de minimizare îşi are rostul ei încercați să desenați circuitul combinațional care realizează funcția logică descrisă de ecuația (13.1).

De multe ori funcția logică trebuie scrisă pornind de la tabelul de adevăr care descrie funcționarea circuitului. În acest caz, pentru a scrie

expresia funcției logice care trebuie realizată, se recomandă parcurgerea următoarelor două etape:

- se scrie câte un termen ŞI pentru fiecare combinație a nivelurilor logice de intrare pentru care ieşirea este la nivel logic 1. Fiecare termen ŞI trebuie să conțină toate variabilele de intrare sub formă inversată sau neinversată după cum în linia corespunzătoare din tabel apar la nivel logic 0 sau 1.
- termenii ŞI astfel obținuți sunt legați între ei cu operația logică SAU, obținându-se expresia finală a funcției logice.
- dacă este necesar, se simplifică funcția logică folosind teoremele algebrei Booleene.

Să considerăm exemplul din Tabelul 13.1 în care avem trei variabile de intrare *A*, *B* şi *C* şi o variabilă de ieşire, *x*. Aplicând regulile de mai sus se obține expresia funcției logice care trebuie realizată.

Tabelul 13.1

C	***B***	***A***	***x***	**Termeni ŞI**	**Funcția logică**
0	0	0	0	-	$x = A\overline{B}\,\overline{C} + \overline{A}B\overline{C} + ABC$
0	0	1	1	$A\overline{B}\,\overline{C}$	
0	1	0	1	$\overline{A}B\overline{C}$	
0	1	1	0	-	
1	0	0	0	-	
1	0	1	0	-	
1	1	0	0	-	
1	1	1	1	ABC	

La o primă observare constatăm că funcția noastră este sub formă canonică minimizată, astfel încât putem trece la proiectarea circuitului logic care să o realizeze. Din analiza ei se poate vedea că avem nevoie de o poartă SAU cu trei intrări, de trei porți ŞI tot cu trei intrări şi de trei inversoare, deoarece toate variabilele de intrare apar şi sub formă inversată. Circuitul logic care realizează funcția este prezentat în fig.13.2.

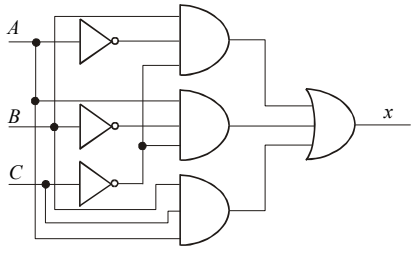


Fig.13.2

Din cele prezentate până acum se poate observa că scrierea unei funcții logice sub formă de sumă de produse facilitează proiectarea circuitului care să o realizeze folosind porțile logice elementare ŞI, SAU şi INVERSORUL. Din punct de vedere teoretic totul pare a fi în ordine. Din punct de vedere practic însă, apare un mic inconvenient. Majoritatea circuitelor integrate care conțin porți logice au la bază porțile logice ŞI-NU şi SAU-NU cu ajutorul cărora se pot realiza toate celelalte funcții logice elementare. Am arătat acest lucru în Capitolul 12. Dacă în procesul de implementare a funcțiilor logice se vor folosi astfel de circuite integrate, este evident necesar ca schema logică ce realizează funcția minimizată să fie realizată numai cu porți ŞI-NU sau cu porți SAU-NU.

13.1.2 Minimizarea cu diagrame Karnaugh

O altă metodă folosită pentru minimizarea funcțiilor logice este cea a *diagramei Karnaugh*. Ea este o metodă grafică de obținere a funcției logice minimizate şi de proiectare circuitul logic care să o realizeze, având ca punct de start tabelul de adevăr. Teoretic, metoda poate fi folosită pentru un număr de variabile de intrare oricât de mare, însă practic este aplicabilă pentru cel mult şase variabile de intrare.

Diagrama Karnaugh este un careu de formă pătratică sau dreptunghiulară conținând 2^N căsuțe, N fiind numărul variabilelor de intrare. Fiecare căsuță corespunde unei singure combinații posibile de formă ŞI a variabilelor de intrare. Atât pe orizontală cât şi pe verticală, două căsuțe adiacente diferă între ele doar prin valoarea logică a unei singure variabile din combinațiile corespunzătoare lor. În fiecare căsuță se va înscrie cifra 1 sau 0 după cum combinația corespunzătoare ei are ca rezultat 1 logic sau 0 logic.

Expresia minimizată a variabilei de ieşire poate fi obținută din diagrama Karnaugh prin gruparea şi încercuirea căsuțelor adiacente care conțin variabila binară 1. Gruparea se poate face în perechi de ***două***, ***patru*** sau ***opt*** căsuțe. Se mai spune că se face gruparea în ***dubleți***, ***quazi*** sau ***octeți***. Trebuie menționat faptul că *se consideră adiacente şi pătratele de la extremitățile unei linii sau unei coloane*.

Să considerăm exemplul din Tabelul 13.2 căruia îi corespunde diagrama Karnaugh din fig. 13.3. În acest exemplu se pot grupa în dubleți căsuțele cu numerele 2 şi 6, respectiv 10 şi 11. Având doi dubleți, expresia finală a funcției logice va avea doi termeni care pot fi obținuți astfel: din primul dublet dispare variabila B care apare atât în forma normală cât şi inversată, astfel că primul termen al funcției va fi $\overline{A}\overline{C}D$; din al doilea dublet dispare variabila C care apare atât în forma normală cât şi inversată,

astfel că al doilea termen al funcţiei va fi *ABD*. Expresia finală funcţiei logice va fi:

$$x = \overline{A}\overline{C}D + ABD$$

Tabelul 13.2

D	***C***	***B***	***A***	***x***	**Termeni ŞI**
0	0	0	0	0	-
0	0	0	1	0	-
0	0	1	0	0	-
0	0	1	1	0	-
0	1	0	0	0	-
0	1	0	1	0	-
0	1	1	0	0	-
0	1	1	1	0	-
1	0	0	0	1	$\overline{A}\overline{B}\overline{C}D$
1	0	0	1	0	-
1	0	1	0	1	$\overline{A}B\overline{C}D$
1	0	1	1	1	$AB\overline{C}D$
1	1	0	0	0	-
1	1	0	1	0	-
1	1	1	0	0	-
1	1	1	1	1	$ABCD$

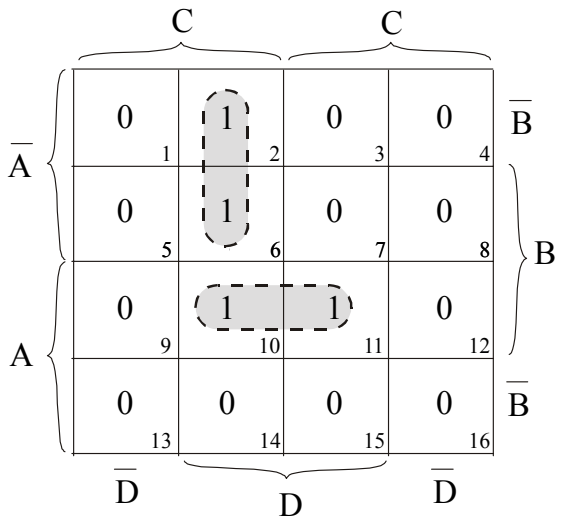


Fig.13.3

Gruparea în quazi o exemplificăm pe diagrama Karnaugh din fig.13.4.

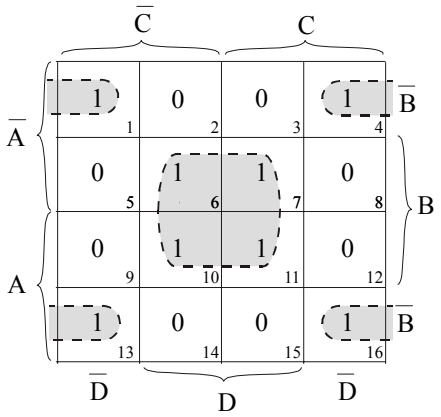


Fig.13.4

Prin gruparea în quazi se elimină câte două variabile din fiecare quad, evident cele care apar în formele normală şi inversată. În exemplul nostru, expresia funcției logice va avea doi termeni de câte două variabile pentru că avem doi quazi. Astfel, din quadul care cuprinde căsuțele 6, 7, 10 şi 11 se elimină variabilele *A* şi *C*, rămânând termenul *BD*, iar din quadul care conține căsuțele din cele patru colțuri se elimină tot variabilele *A* şi *C*, rămânând termenul $\overline{B}\overline{D}$. Astfel, expresia funcției logice realizate va fi:

$x = BD + \overline{B}\overline{D}$

În cazul grupării în octeți se aplică aceleaşi reguli, cu deosebirea că prezența unui octet este echivalentă cu eliminarea a trei variabile din termenul corespunzător lui.

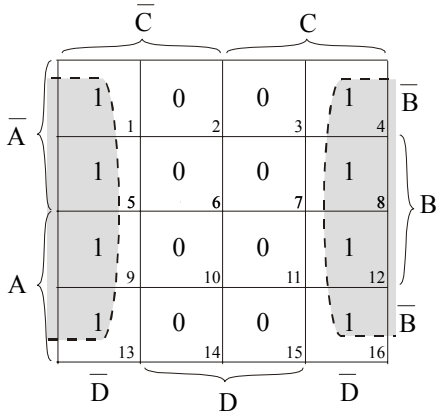


Fig.13.5

În cazul exemplului din fig.13.5 se elimină variabilele *A*, *B* şi *C*. Având un singur octet, funcţia logică va avea un singur termen şi expresia ei va fi:

$x = \overline{D}$

Unele circuite pot fi proiectate astfel încât să existe anumite stări ale intrărilor pentru care nivelul logic al ieşirii să nu fie precizat, pentru simplul motiv ca stările respective ale intrărilor nu se vor realiza niciodată în situaţia concretă de funcţionare a circuitului. Acestea se numesc *stări nedeterminate*. În aceste situaţii, proiectantul are libertatea de a pune în căsuţele corespunzătoare stărilor de nedeterminare *0* sau *1* astfel încât să-i fie cât mai uşor să simplifice expresia booleană a funcţiei de ieşire.

Este foarte probabil ca în multe cazuri să nu putem grupa căsuţele dintr-o diagramă Karnaugh numai în dubleţi, quazi sau octeţi, având şi situaţii mai complexe în care va trebui să lucrăm pe aceeaşi diagramă cu două, trei sau chiar patru tipuri de grupări. Când am spus patru, ne-am gândit şi la cazurile de termeni izolaţi care nu pot fi grupaţi cu alţi termeni. În aceste situaţii se recomandă parcurgerea următoarei succesiuni de paşi pentru obţinerea formei finale a funcţiei logice:

- construirea diagramei Karnaugh pe baza tabelului de adevăr. Este important de menţionat că dacă există combinaţii ale variabilelor de intrare pentru care starea ieşirii este nedeterminată (ea poate fi 0 sau 1), proiectantul are libertatea ca în diagrama Karnaugh, în căsuţa corespunzătoare combinaţiilor respective să pună 0 sau 1, astfel încât aceasta să-l ajute la minimizarea mai eficientă a funcţiei.
- se vor încercui căsuţele izolate care conţin variabila 1. Aceste căsuţe nu sunt adiacente cu alte căsuţe care conţin variabila binară 1.
- se vor căuta căsuţele care conţin variabila 1 şi care au o singură căsuţă adiacentă care conţine variabila 1. Astfel se realizează dubleţii.
- se încercuiesc octeţii chiar dacă vreo căsuţă din ei a fost inclusă în dubleţi.
- se încercuiesc quazii chiar dacă vreo căsuţă din ei a fost inclusă în dubleţi sau octeţi.
- se încercuieşte orice pereche care include căsuţe care încă nu au fost încercuite, asigurându-ne că numărul de încercuiri este minim.

- se face suma termenilor generați de fiecare grupare, obținându-se astfel expresia finală a funcției logice.

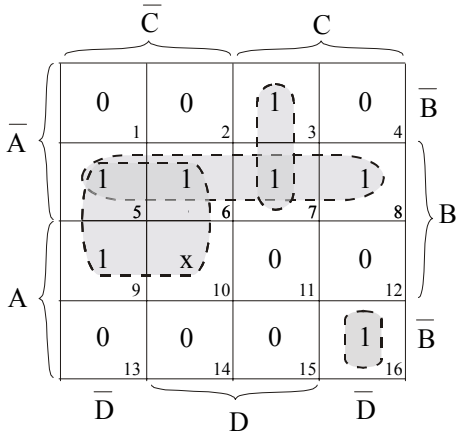


Fig.13.6

Să aplicăm în ordine aceste reguli pe exemplul din fig.13.6:

1. - căsuța 10 prezintă o stare de nedeterminare şi ne avantajează să o considerăm în starea 1.

2. - căsuța 16 este izolată şi ea va genera termenul $A\overline{B}C\overline{D}$.

3. - căsuța 3 se învecinează numai cu căsuța 7 formând dubletul 3-7 care generează termenul $\overline{A}CD$.

4. - nu sunt octeți.

5. - există doi quazi: 5-6-7-8 care generează termenul $\overline{A}B$ *şi 5-6-9-10 care generează termenul* $B\overline{C}$.

6. - nu mai sunt alte perechi şi căsuțe conținând 1, neincluse în combinațiile precedente.

7. - expresia finală a funcției logice va fi:

$$x = A\overline{B}C\overline{D} + \overline{A}CD + \overline{A}B + B\overline{C}$$

13.2 Porțile SAU EXCLUSIV şi SAU EXCLUSIV-NU

Poarta SAU EXCLUSIV este o poartă logică care poate avea numai două intrări şi care

furnizează un 1 logic la ieşire ori de câte ori cele două intrări sunt în stări complementare.

Simbolul operației SAU EXCLUSIV este "$\oplus$". Să proiectăm circuitul logic care o realizează pornind de la scrierea tabelului de adevăr conform afirmației precedente:

Tabelul 13.3

B	***A***	$x = A \oplus B$	**Termeni ŞI**
0	0	0	
0	1	1	$A\overline{B}$
1	0	1	$\overline{A}B$
1	1	0	

şi de la expresia funcției logice care realizează operația SAU EXCLUSIV, scrisă pe baza lui:

$$A \oplus B = A\overline{B} + \overline{A}B$$

Observăm imediat că avem nevoie de două inversoare, două porți ŞI şi o poartă SAU, astfel încât circuitul logic combinațional pentru funcția noastră arată ca cel din fig.13.7, în care este prezentat şi simbolul porții SAU EXCLUSIV folosit în schemele digitale.

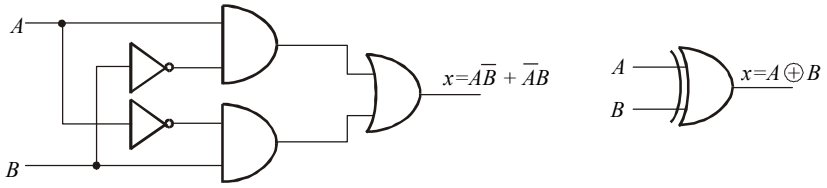


Fig.13.7

Poarta SAU EXCLUSIV-NU operează exact în opoziție cu poarta SAU EXCLUSIV. Tabelul 13.4 ne ajută la scrierea expresiei funcției logice pentru această operație:

Tabelul 13.4

B	***A***	$\overline{x = A \oplus B}$	**Termeni ŞI**
0	0	1	$\overline{A}\overline{B}$
0	1	0	
1	0	0	
1	1	1	AB

$$x = AB + \overline{A}\overline{B}$$

Ea ne indică faptul că variabila *x* va avea valoarea logică 1 în două cazuri: *A = B = 1* (termenul *AB*) şi *A = B = 0* (termenul $\overline{A}\overline{B}$). Cu alte cuvinte:

poarta SAU EXCLUSIV-NU va produce un nivel înalt al tensiunii de ieşire ori de câte ori cele două intrări vor fi la acelaşi nivel logic.

Deoarece această poartă compară două niveluri logice şi "ne atrage atenția" când ele sunt egale, se mai spune că ea realizează funcția de *echivalență*. În fig.13.8 este arătat circuitul cu ajutorul căruia poate fi realizată această funcție logică şi simbolul porții logice aferente.

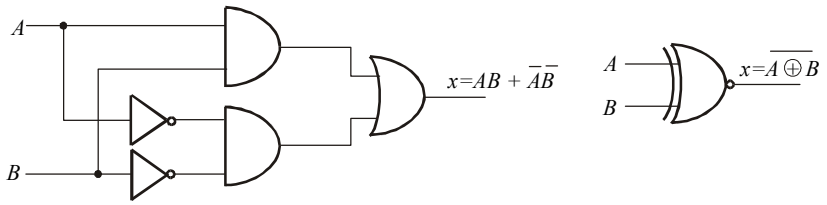


Fig.13.8

Deoarece circuitul SAU EXCLUSIV-NU operează complementar cu circuitul SAU EXCLUSIV, simbolul să poate fi obținut din simbolul porții SAU EXCLUSIV prin adăugarea cerculeţului de negație la ieşire.

13.3 Circuite pentru prelucrarea informațiilor digitale

Pentru prelucrarea datelor în sistemele digitale şi apoi pentru citirea şi afişarea rezultatelor prelucrării, sunt necesare mai multe etape de lucru:

- *codarea* şi *decodarea* (transformarea datelor dintr-un cod în altul)
- *multiplexarea* (transmiterea către o ieşire a unei singure informații dintr-un grup de informații)
- *demultiplexarea* (introducerea succesivă a datelor la diferite adrese posibile)

Toate aceste operații pot fi realizate cu ajutorul porților logice conectate în combinații rezultate în urma stabilirii funcției (funcțiilor) logice de transfer pe care trebuie să o (le) realizeze circuitul.

13.3.1 Circuite de codare a informației

Un circuit de codare are un anumit număr de intrări (codul de intrare), dintre care doar una poate fi activată la un moment dat şi *N* ieşiri care reprezintă numărul de biți ai codului în care sunt reprezentate informațiile de la intrare. La un circuit de codare numărul de biți ai codului de ieşire este mai mic decât numărul de biți ai codului de intrare. Cel mai frecvent caz este acela al codării în binar. În această situație:

$N = log_2$(numărul de intrări)

Pentru a exemplifica modalitatea de proiectare circuitelor de codare să considerăm exemplul unui circuit de codare cu opt intrări şi $N = log_2 8 = 3$ ieşiri. Să notăm cu A_0, A_1, … A_7 cele opt intrări şi cu O_0, O_1 şi O_2 cele trei ieşiri şi să construim un tabel de adevăr în care *combinaţia biţilor de la ieşire să fie corespondentul binar al indicelui zecimal intrării*.

Tabelul 13.5

A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	O_2	O_1	O_0
0	0	0	0	0	0	0	**1**	0	0	0
0	0	0	0	0	0	**1**	0	0	0	1
0	0	0	0	0	**1**	0	0	0	1	0
0	0	0	0	**1**	0	0	0	0	1	1
0	0	0	**1**	0	0	0	0	1	0	0
0	0	**1**	0	0	0	0	0	1	0	1
0	**1**	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Încercând să stabilim o corespondenţă biunivocă între stările logice ale ieşirilor şi cele ale intrărilor, vom observa că:

O_0 = 1 dacă A_1 *SAU* A_3 *SAU* A_5 *SAU* A_7 sunt la nivel logic 1
O_1 = 1 dacă A_2 *SAU* A_3 *SAU* A_6 *SAU* A_7 sunt la nivel logic 1
O_2 = 1 dacă A_4 *SAU* A_5 *SAU* A_6 *SAU* A_7 sunt la nivel logic 1

Deci, circuitul de codare va trebui să aibă câte o poartă SAU cu patru intrări care să comande fiecare ieşire. Modul de conectare a intrărilor circuitului de codare la intrările celor patru porţi SAU este arătat în fig.13.9. Intrarea A_0 nu este conectată deoarece ieşirea va indica automat starea 000 dacă $A_1 = A_2 = \ldots = A_7 = 0$.

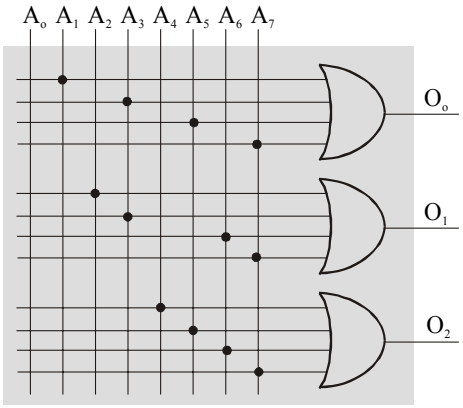


Fig.13.9

Unul dintre neajunsurile circuitului de codare, aşa cum este el prezentat în fig.13.9, este acela că dacă două intrări sunt simultan la nivel logic 1, atunci rezultatul este eronat. De exemplu, dacă intrările A_3 şi A_5 sunt simultan la nivel logic 1, atunci stările ieşirilor vor fi 111, ceea ce corespunde nivelului logic 1 la intrarea A_7. De aceea au fost realizate *circuite de codare cu prioritate,* care conțin circuite logice astfel aranjate încât dacă două sau mai multe intrări sunt aduse simultan la nivel logic 1, atunci la ieşire va avea prioritate (va apare) codul numărului mai mare de la intrare.

13.3.2 Circuite de decodare a informației

Operația inversă a codării este decodarea. Un decodor este un circuit logic combinațional cu N intrări *şi* $M \leq 2^N$ ieşiri. La intrarea decodorului se aplică o informație codată pe N biți. Pentru o combinație dată a nivelurilor logice de la intrare va fi activată o singură ieşire. Deoarece unele coduri nu folosesc toate combinațiile posibile ale nivelurilor logice oferite de numărul de biți pe care este exprimată informația, numărul de ieşiri poate fi şi mai mic decât 2^N. Astfel, când o informație zecimală este codată în binar (BCD)se folosesc numai 10 (0000, … , 1001), din cele 16 combinații posibile deci un decodor BCD $\rightarrow$ zecimal nu va avea 16 ieşiri ci numai 10.

Unul dintre cele mai folosite decodoare este cel de la 3 la 8 linii. Proiectarea lui cu porți logice poate fi realizată dacă se cunoaşte funcția de transfer pentru fiecare ieşire. Aceasta poate fi exprimată pe baza tabelului de adevăr 13.6.

Tabelul 13.6

A_2	A_1	A_0	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0	**Funcția de transfer**
0	0	0	0	0	0	0	0	0	0	**1**	$O_0 = \overline{A_2}\,\overline{A_1}\,\overline{A_o}$
0	0	1	0	0	0	0	0	0	**1**	0	$O_1 = \overline{A_2}\,\overline{A_1}\,A_o$
0	1	0	0	0	0	0	0	**1**	0	0	$O_2 = \overline{A_2}\,A_1\,\overline{A_o}$
0	1	1	0	0	0	0	**1**	0	0	0	$O_3 = \overline{A_2}\,A_1\,A_o$
1	0	0	0	0	0	**1**	0	0	0	0	$O_4 = A_2\,\overline{A_1}\,\overline{A_o}$
1	0	1	0	0	**1**	0	0	0	0	0	$O_5 = A_2\,\overline{A_1}\,A_o$
1	1	0	0	**1**	0	0	0	0	0	0	$O_6 = A_2\,A_1\,\overline{A_o}$
1	1	1	**1**	0	0	0	0	0	0	0	$O_7 = A_2\,A_1\,A_o$

Cunoscând funcţiile de transfer pentru fiecare ieşire, putem trece la proiectarea circuitului de decodificare a informaţiei. Se vede că fiecare ieşire este caracterizată de un produs de trei termeni în stare normală sau complementară. Deci, pentru fiecare ieşire vom folosi câte o poartă ŞI cu cel puţin trei intrări. Unele decodoare au şi una sau mai multe intrări de validare cu ajutorul cărora se poate controla starea de funcţionare a lor. Astfel, dacă pentru decodorul de la 3 la 8 linii se folosesc porţi ŞI cu patru intrări (fig.13.10), cea de-a patra intrare a fiecăreia dintre porţi poate fi folosită ca intrare de validare, *E* (***E****NABLE*).

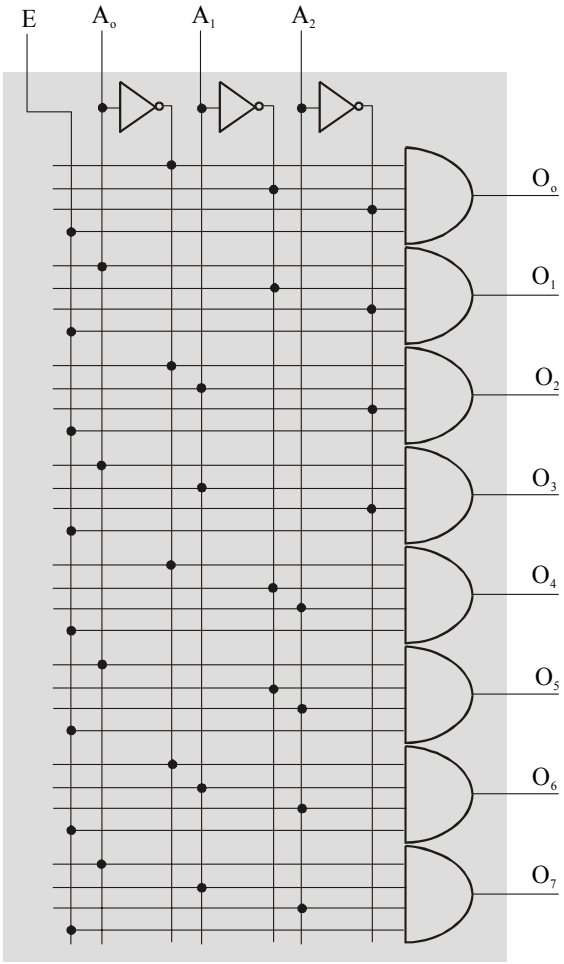


Fig.13.10

Se poate observa că dacă *E* =1, decodorul va funcţiona iar dacă *E* = 0 el va fi blocat.

13.3.3 Multiplexoare

Un multiplexor este un circuit logic combinațional cu mai multe intrări şi o singură ieşire. El acceptă mai multe date de intrare, permițând doar uneia dintre ele să treacă la un moment dat spre ieşire. Deoarece face o selecție de date, multiplexorul mai este denumit *SELECTOR DE DATE*. Ordinea de transmitere a datelor spre ieşire este hotărâtă de una sau mai multe intrări de dirijare a informației, numite **intrări de selecție**. Dacă vreți, putem compara multiplexorul cu o gară cu mai multe linii pe care se află trenuri care trebuie să o părăsească într-o anumită ordine, între două gări existând o singură linie.

Multiplexorul acționează ca un comutator multipozițional controlat digital, în care codul digital aplicat la intrarea de selecție hotărăşte care este ordinea de transmitere spre ieşire a datele de intrare (fig.13.11). Cu alte cuvinte, multiplexorul poate trimite la pensie bătrânul acar din vechile gări de pe vremea locomotivelor cu aburi.

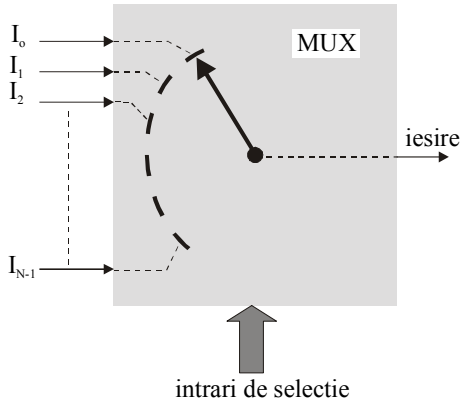


Fig.13.11

Deoarece numărul de stări logice distincte ale intrării de selecție trebuie să fie egal cu numărul de intrări de date *N*, numărul de intrări de selecție poate fi calculat din relația:

$$\text{numărul intrărilor de selecție} = log_2 N$$

Multiplexorul de bază este şi cel mai simplu, având două intrări de date şi o intrare de selecție ($log_2 2 = 1$). Funcția de transfer a multiplexorului poate fi scrisă pe baza tabelului de adevăr 13.7 ca o sumă de produse a termenilor care furnizează un 1 logic la ieşire, tabel în care variabilele de intrare sunt I_o, I_1 şi S (selecție) iar variabila de ieşire este z. Condiția impusă este aceea ca la ieşire să fie transferată informația de la intrarea I_o dacă $S = 0$ şi cea de la intrarea I_1 dacă $S = 1$.

Tabelul 13.7

S	I_1	I_o	z	Termeni ŞI
0	0	0	0	
0	0	1	1	$\overline{S}I_o\overline{I_1}$
0	1	0	0	
0	1	1	1	$\overline{S}I_oI_1$
1	0	0	0	
1	0	1	0	
1	1	0	1	$S\overline{I_o}I_1$
1	1	1	1	SI_0I_1

Funcția de transfer va fi:

$$z = \overline{S}I_o\overline{I_1} + \overline{S}I_oI_1 + S\overline{I_o}I_1 + SI_0I_1$$

care, după minimizare, devine:

$$z = \overline{S}I_o + SI_1$$

Se poate vedea că pentru realizarea ei avem nevoie de două porți ŞI, o poartă SAU şi un INVERSOR, conectate ca în fig.13.12.

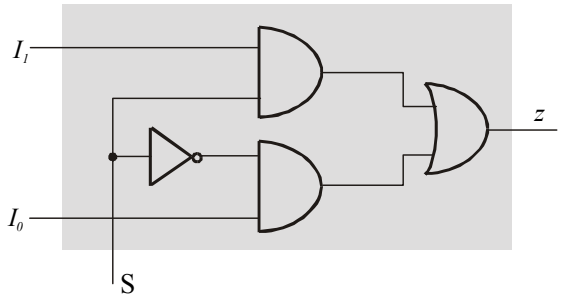


Fig.13.12

În mod analog, pot fi gândite scheme de multiplexoare cu patru, opt sau şaisprezece intrări, multiplexoare care sunt realizate sub formă integrată.

13.3.4 Demultiplexoare

Operația inversă multiplexării este demultiplexarea. De data aceasta trenurile nu mai ies din gară ci intră în ea pe rând, pe o singură linie, şi trebuie distribuite pe liniile gării. Această operație o face demultiplexorul (fig.13.13). Deoarece numărul de stări logice distincte ale intrării de selecție trebuie să fie egal cu numărul de ieşiri de date N, numărul de intrări de selecție poate fi calculat din relația:

$N = 2^{\text{numărul de intrări de selecție}}$

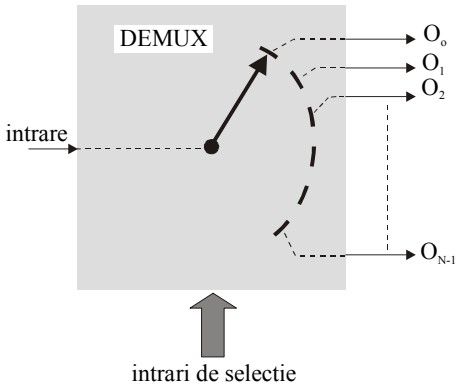


Fig.13.13

Proiectarea unui demultiplexor se poate face stabilind funcția de transfer pe baza tabelului de adevăr. Astfel, dacă avem o gară cu opt linii, multiplexorul care distribuie trenurile va trebui să aibă trei intrări de selecție ($log_2 8 = 3$). Variabilele de intrare în demultiplexor vor fi cele de la intrarea de date şi intrările de selecție. Punând condiția ca primele opt date de intrare să fie distribuite în ordine la cele opt ieşiri, se poate construi următorul tabel de adevăr:

Tabelul 13.8

S_2	S_1	S_o	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0	**Funcția de transfer**
0	0	0	0	0	0	0	0	0	0	**I**	$O_0 = I\ \overline{S_2}\,\overline{S_1}\,\overline{S_o}$
0	0	1	0	0	0	0	0	0	**I**	0	$O_1 = I\ \overline{S_2}\,\overline{S_1}\,S_o$
0	1	0	0	0	0	0	0	**I**	0	0	$O_2 = I\ \overline{S_2}\,S_1\,\overline{S_o}$
0	1	1	0	0	0	0	**I**	0	0	0	$O_3 = I\ \overline{S_2}\,S_1\,S_o$
1	0	0	0	0	0	**I**	0	0	0	0	$O_4 = I\ S_2\,\overline{S_1}\,\overline{S_o}$
1	0	1	0	0	**I**	0	0	0	0	0	$O_5 = I\ S_2\,\overline{S_1}\,S_o$
1	1	0	0	**I**	0	0	0	0	0	0	$O_6 = I\ S_2\,S_1\,\overline{S_o}$
1	1	1	**I**	0	0	0	0	0	0	0	$O_7 = I\ S_2\,S_1\,S_o$

După cum se poate vedea, pentru codul de selecție 000 valoarea logică a intrării *I* este dirijată către ieşirea O_0, pentru codul de selecție 001

valoarea logică a intrării *I* este dirijată către ieşirea O_1, … , pentru codul de selecţie 111 valoarea logică a intrării *I* este dirijată către ieşirea O_7. Deci, demultiplexorul poate fi construit din opt porţi ŞI cu câte patru intrări, câte o poartă pentru fiecare ieşire. De asemenea, mai sunt necesare trei inversoare, câte unul pentru fiecare intrare de selecţie. Schema de principiu a acestui multiplexor este arătată în fig.13.14.

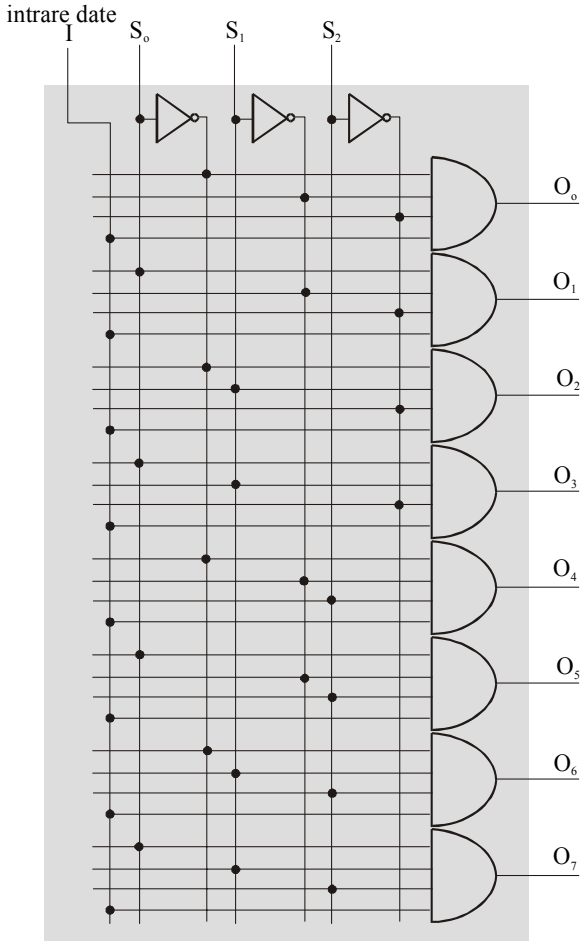


Fig.13.14

O analiză atentă a schemei demultiplexorului ne va arăta că ea este identică cu aceea a unui decodor cu o intrare de validare. Pentru a fi folosit ca demultiplexor, intrările decodorului sunt folosite ca intrări de selecţie iar intrarea de validare este folosită ca intrare de date. Pentru că pot fi folosite în ambele scopuri, circuitele integrate de acest tip sunt denumite DECODOARE/ DEMULTIPLEXOARE.